Pairs Trading on Borsa Istanbul

Berk Aydemir
Buse Bek
Egemen Ayraç
Evren Ataş
Tuana Ilgın Aksoy

Pairs Trading: Performance of a Relative-Value Arbitrage Rule

A Comprehensive Summary

Pairs Trading: Performance of a Relative Value Arbitrage Rule

Evan Gatev
Simon Fraser University

William N. Goetzmann
Yale School of Management - International Center for Finance; National Bureau of Economic Research (NBER)

K. Geert Rouwenhorst
Yale School of Management - International Center for Finance

February 2006

Yale ICF Working Paper No. 08-03

Abstract:
We test a Wall Street investment strategy, pairs trading, with daily data over 1962-2002. Stocks are matched into pairs with minimum distance between normalized historical prices. A simple trading rule yields average annualized excess returns of up to 11 percent for selffinancing portfolios of pairs. The profits typically exceed conservative transaction costs estimates. Bootstrap results suggest that the pairs effect differs from previously-documented reversal profits. Robustness of the excess returns indicates that pairs trading profits from temporary mis-pricing of close substitutes. We link the profitability to the presence of a common factor in the returns, different from conventional risk measures.

The Anatomy of the Paper

Question: If two stocks move together for a given period of time, do temporary divergences create a profit opportunity?

The Hypothesis: Pairs trading is a relative-value statistical arbitrage strategy that profits from the temporary mispricing of close equity substitutes. It is rooted in the "law of one price".

The Mechanism: Stocks are matched based on historical price movements (Sum of Squared Differences), identifying pairs that historically move together.

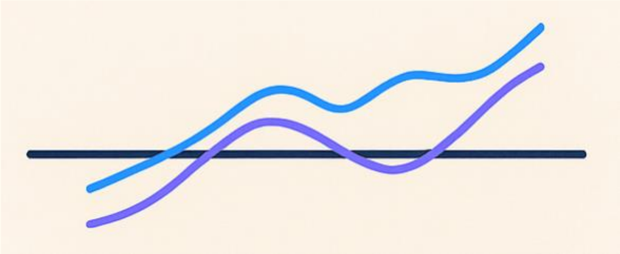
The Signals: The strategy opens a long-short position when prices diverge by two standard deviations, and closes the position upon convergence.

The Strategy

Pairs trading is a quantitative, market-neutral investment strategy utilized by hedge funds.

When the spread between two historically correlated assets widens beyond a norm, the arbitrageur sells the outperforming stock and buys the underperforming stock.

The theory behind it exploits the **Law of One Price**. In an efficient market, securities representing close economic substitutes must have nearly identical prices. The arbitrageur profits when the temporary divergence corrects itself and the prices converge back to equilibrium.



Pair Formation

Prevention of Look-Ahead Bias

- **Stage 1: The Formation Period (12 Months)**

A rolling window used strictly to observe historical price behaviours.

- **Stage 2: The Trading Period (6 Months)**

A non-overlapping, out-of-sample window used for execution.

- **Rolling Implementation**

Pair Formation

01 Liquidity Screening

- The algorithm filters the universe of equities to remove any stock with missing trading days during the formation period.

02 Price Normalization

- Every stock's cumulative total return index (including reinvested dividends) is normalized to starting value of 1.0 at the beginning of the formation period.

03 Calculating SSD

- For every possible combination of stocks, calculate the Sum of Squared Differences (SSD) between daily normalized prices.

$$\mathrm{SSD}_{A,B} = \sum_t (P^*_{A,t} - P^*_{B,t})^2$$

04 Ranking and Selection

- Pairs are ranked by SSD from lowest to highest. The study focuses on the "TOP 5" and "TOP 20" closest economic substitutes.

Trading Window

The strategy opens a long-short position when prices diverge by two standard deviations. The standard deviation is estimated historically based on data from the 12 month pairs formation period.

Entry Signal: A long-short position is opened immediately when the normalized prices diverge by more than two historical standard deviations.

$$|P_A^* - P_B^*| > 2\sigma_{\text{formation}}$$

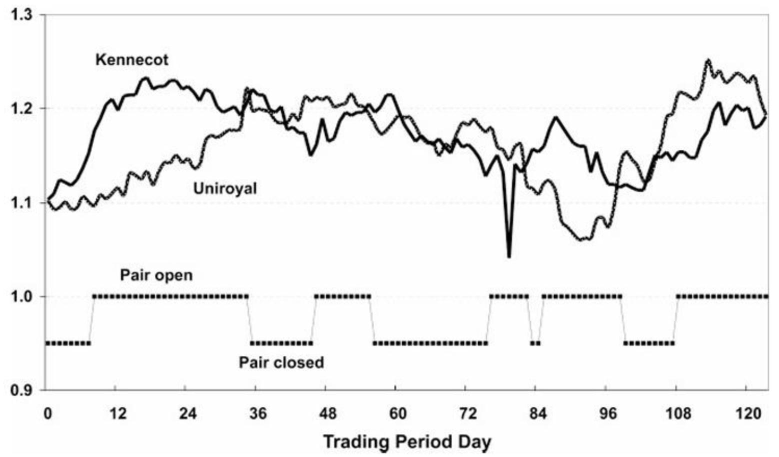


Figure 1
Daily normalized prices: Kennecott and Uniroyal (pair 5)
Trading period August 1963–January 1964.

Trading Window

The Position Sizing: The trade is executed as a dollar-neutral portfolio. The algorithm goes one dollar short in the higher-priced stocks and one dollar long in the lower-priced stock.

Exit Signal: The position is closed at the exact moment the two normalized prices cross again.

$$P_A^* = P_B^*$$

- ➔ If the prices fail to converge during the 6-month trading period, the position is automatically closed on the final day.

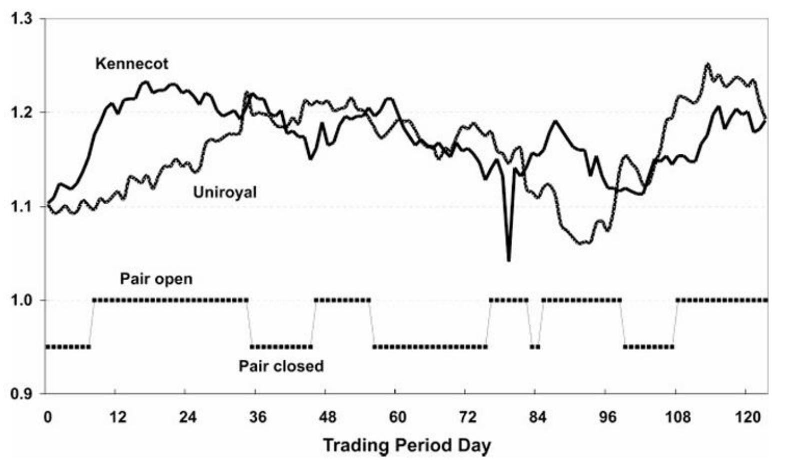


Figure 1
Daily normalized prices: Kennecott and Uniroyal (pair 5)
Trading period August 1963–January 1964.

The Results

Analyzing daily data from 1962 to 2002, a fully invested portfolio of the top 20 closest pairs earned roughly 11% annualized over the 40 year sample period. This is equal to a 1.44% monthly excess return.

1962-2002 | **11% Annualized Return** | **1.44% Annualized Monthly Excess Return**

Top 20 Pairs: Performance Summary

Metric	Value
Annualized Excess Return	11.0%
Sharpe Ratio	1.25
Win Rate (Monthly)	62%
Max Drawdown	-8.5%
t-Statistic	>11.0

Returns are significant with a t-static exceeding 11%.

The strategy exhibits positive skewness in returns. The worst monthly loss was bounded and the strategy yielded positive cash flows with a high frequency of winning trades.

The strategy works best in times of uncertainty.

The Results

- **Problem:** Pairs trading is a contrarian strategy.

Raw returns may be artificially inflated by the bid-ask bounce.

- **Solution:** Introducing a one-day delay.

Positions are opened the day after divergence occurs and closed the day after convergence occurs.

- **Result:** Monthly excess return drops from 1.44% to 0.90%. This return is still economically and statistically significant.
- The strategy also survives real-world trading costs with a net profit ranging from 113 to 225 basis points, even after subtracting 324 basis points in total transaction costs per 6 month period.

Pairs Trading and Standard Factor Investing

Paris trading can’t be explained by standard factor investing:

Beta is near zero. The strategy exhibits insignificant exposure to size (SMB) and value (HML). It is truly market-neutral.

It has negative exposure to momentum but it is not enough to explain the profits.

Bootstrapping random pairs proves that it captures a unique relative-value dynamic beyond 1 month mean reversion.

Alpha remains highly significant across all regressions. The profits can’t be explained by standard risk models.

Performance decay: Before and after 1989.

Is it worth investing?

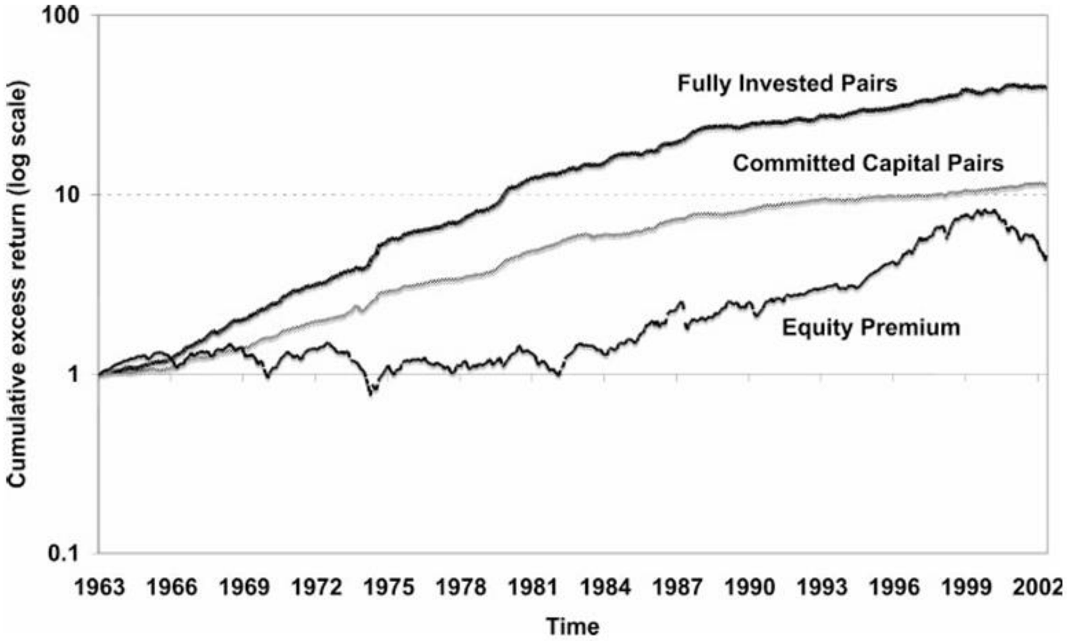


Figure 3
Cumulative excess return of top 20 pairs and S&P 500
May 1963–December 2002.

Liquidity Filtering

```python
def liquidity_filter(
    prices: pd.DataFrame,
    formation_start,
    formation_end,
) -> list:
    """Return tickers with complete price histories during the formation window.

    GGR (2006) §2: "We restrict the sample to stocks with complete return
    histories during the formation period." The criterion is price completeness
    only — any ticker with a single missing price observation is excluded.
    The paper does not apply a volume screen.

    Parameters
    ----------
    prices              : full-history price DataFrame; columns = ticker names.
    formation_start/end : inclusive date bounds of the 12-month window.

    Returns
    -------
    list of tickers that have no NaN prices in the formation window.
    """
    px_w = prices.loc[formation_start:formation_end]
    return [t for t in px_w.columns if px_w[t].notna().all()]
```

In each formation window, a stock with $>=$ 1 day with no trade is removed.

SSD Computation

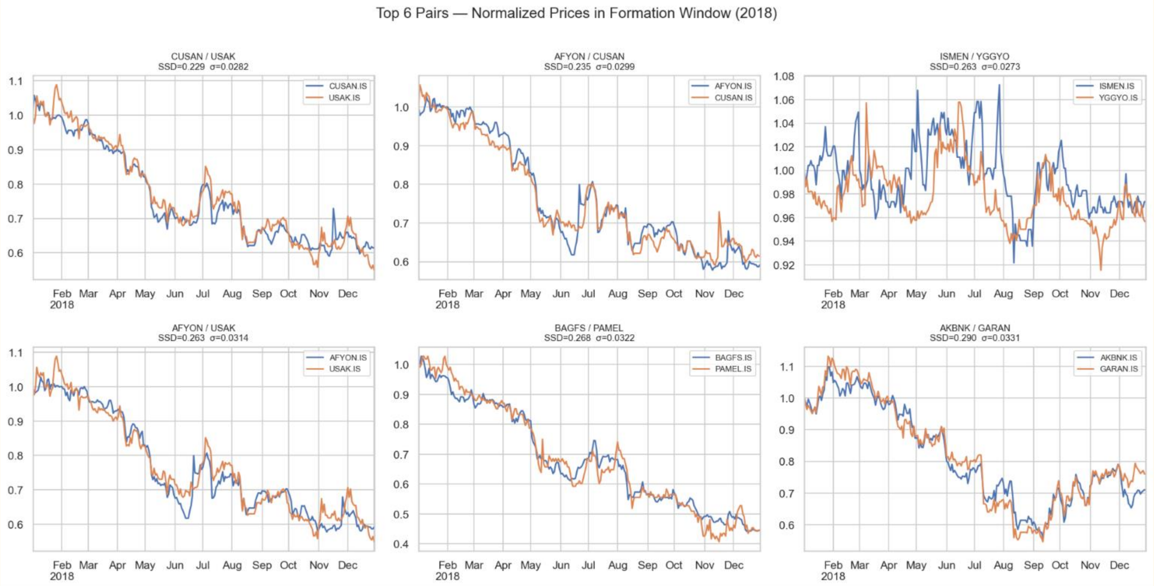
```python
def compute_ssd(norm_prices: pd.DataFrame) -> pd.DataFrame:
    """Compute SSD_{A,B} = Σ_t (P*_{A,t} - P*_{B,t})² for all N(N-1)/2 pairs.

    Vectorized via matrix algebra so it scales to large universes (N=200+) without
    Python loops. NaN values are treated as 0 contributions to the dot product,
    which is consistent with nansum semantics.

    Returns pd.DataFrame with columns [ticker1, ticker2, ssd], sorted ascending.
    """
    tickers = norm_prices.columns.tolist()
    arr = norm_prices.values.astype(float)
    arr_filled = np.where(np.isnan(arr), 0.0, arr)

    gram = arr_filled.T @ arr_filled                    # (N, N)
    sq_sum = np.nansum(arr ** 2, axis=0)                # (N,)
    ssd_matrix = sq_sum[:, None] + sq_sum[None, :] - 2 * gram  # (N, N)

    rows, cols = np.triu_indices(len(tickers), k=1)
    ssd_vals = ssd_matrix[rows, cols]
    order = np.argsort(ssd_vals)
    return pd.DataFrame({
        "ticker1": [tickers[rows[i]] for i in order],
        "ticker2": [tickers[cols[i]] for i in order],
        "ssd":     ssd_vals[order],
    }).reset_index(drop=True)
```

Each pair among N choose 2, they are ranked according to their SSD.

The lower the SSD, the higher the rank.

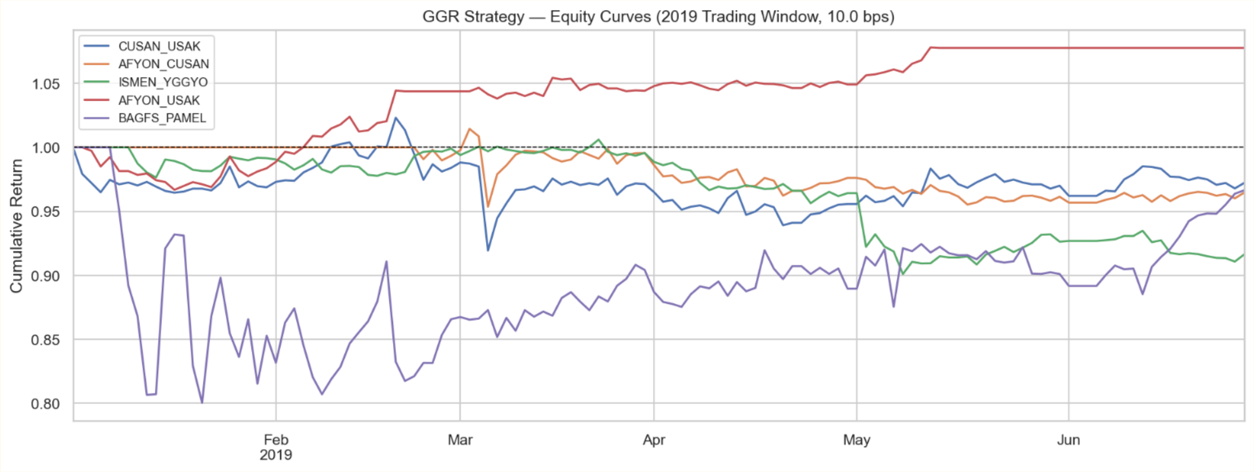
Example: Formation in January 2018-January 2019

rank	ticker1	ticker2	SSD	locked_sigma	entry_threshold_2sigma
1	CUSAN	USAK	0.2294	0.02817	0.05635
2	AFYON	CUSAN	0.2347	0.02987	0.05974
3	ISMEN	YGGYO	0.2631	0.02727	0.05453
4	AFYON	USAK	0.2635	0.03136	0.06271
5	BAGFS	PAMEL	0.2681	0.03218	0.06435
6	AKBNK	GARAN	0.2903	0.03305	0.0661



Example: Trading in January 2019-June 2019

	Sharpe	Return %	Max DD %	Trades
CUSAN_USAK	-0.292	-2.79	10.14	0.0
AFYON_CUSAN	-0.606	-3.55	6.0	0.0
ISMEN_YGGYO	-1.722	-8.37	10.44	0.0
AFYON_USAK	2.109	7.77	3.32	4.0
BAGFS_PAMEL	0.057	-3.36	19.96	0.0



Pair Ranking

```python
def select_top_pairs(ssd_df: pd.DataFrame, n: int = 20, offset: int = 0) -> list:
    """Return n pairs starting at rank `offset` as (ticker1, ticker2, ssd) tuples.

    offset=0   → ranks 1–20    (Top-20, GGR primary result)
    offset=100 → ranks 101–120 (GGR control group: no longer profitable)
    """
    section = ssd_df.iloc[offset: offset + n]
    return list(zip(section["ticker1"], section["ticker2"], section["ssd"]))


def compute_locked_sigma(norm_prices: pd.DataFrame, pairs: list) -> dict:
    """Compute σ = std(P*_A − P*_B) for each pair over the formation window.

    This value is frozen at the end of formation and used unchanged as the
    divergence threshold throughout the 6-month trading period.

    Returns dict keyed by (ticker1, ticker2) → float σ.
    """
    sigmas = {}
    for item in pairs:
        t1, t2 = item[0], item[1]
        if t1 not in norm_prices.columns or t2 not in norm_prices.columns:
            continue
        spread = norm_prices[t1] - norm_prices[t2]
        sigmas[(t1, t2)] = float(spread.std())
    return sigmas
```

Top 20 and pairs from 101st to 120th are selected.

Within each window, their spread is calculated, determining the “threshold signal” for entry.

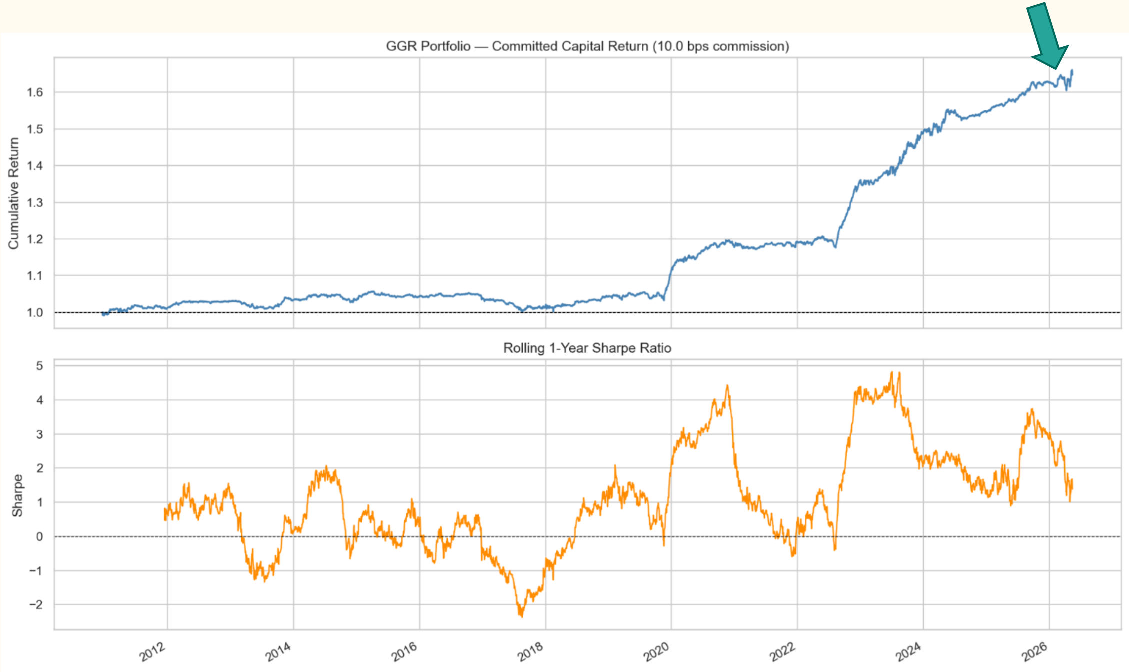
Backtrader Execution: Long, Short, Exit

```python
def _execute_entry(self, direction: int) -> None:
    p1 = self.data0.close[0]
    p2 = self.data1.close[0]
    if p1 <= 0 or p2 <= 0:
        return
    s1 = int(self.p.cash_per_leg / p1)
    s2 = int(self.p.cash_per_leg / p2)
    if s1 == 0 or s2 == 0:
        return
    if direction == -1:
        self.sell(data=self.data0, size=s1)
        self.buy(data=self.data1, size=s2)
        self._in_short = True
    else:
        self.buy(data=self.data0, size=s1)
        self.sell(data=self.data1, size=s2)
        self._in_long = True

def next(self):
    spread = self._spread()
    threshold = self.p.entry_sigma * self.p.locked_sigma

    # Execute orders that were queued on the previous bar (Panel B delay)
    if self._pending_exit:
        self.close(data=self.data0)
        self.close(data=self.data1)
        self._in_long = False
        self._in_short = False
        self._pending_exit = False
        return  # don't generate new signals on the same bar as exit fill
```

For a specific pair,
Position opens when spread $>= 2\sigma$

Same as in the Statistical Arbitrage HW.

Walk Forward Optimization



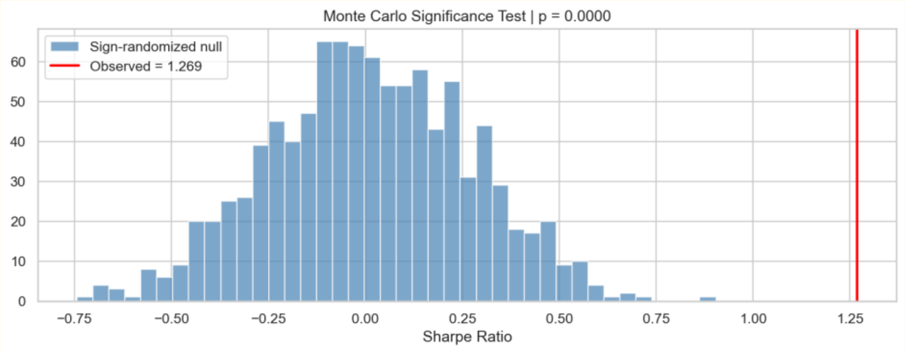
From 2010 to 2026, for each pair we calculated “sigma threshold” in the 12 month formation period.

Now, with respect to this threshold, we trade our pair within following 6 months.

To avoid overfitting, both formation and trading period are shifted by 1 month and procedure is repeated.

At the end, we get a curve by summing “excess returns” from 2010 to 2026.

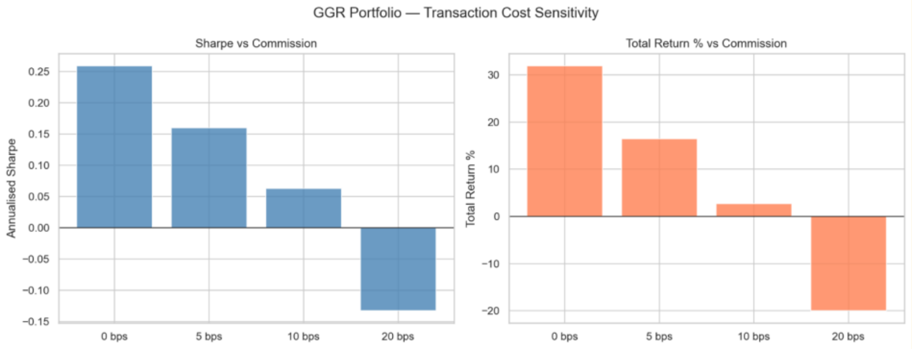
Skill or Luck: Monte-Carlo Significance Test



Paper tests whether the pair's Sharpe ratio significantly differs from a bootstrapped null distribution. By implementing the same logic, we test how often a randomly sign-randomized portfolio's Sharpe exceeds the observed Sharpe, in 1000 tries. This ratio is the 'p-value'.

Effect of Commissions

commission_bps	sharpe	total_return_pct	max_drawdown_pct
0	0.259	31.9	32.3
5	0.16	16.4	32.9
10	0.063	2.7	33.4
20	-0.133	-20.0	41.0



The paper never sets a fixed number. It runs the strategy twice: Trading on the signal day vs one day later. The gap shows the cost (~0.8% spread).

Profit still survives → result is **cost-robust ("invariant").**

We have no bid-ask prices on BIST, so we approximate cost in two ways:

- **Trade one day later** (signal today, buy/sell tomorrow) Adds a realistic cost, same idea as Panel B.
- **Add commission:** charge per trade in bps, tested at 0 / 5 / 10 / 20 to find where profit hits zero.

One pair round-trip = 4 trades, so even small costs add up fast.

Unlike the paper, our BIST profit does NOT survive. Sharpe drops to 0.06 by 10 bps and turns negative at 20 bps.

Cointegration Approach

```python
def select_cointegrated_pairs(
    prices_window: pd.DataFrame,
    candidates: list,
    n: int = 20,
    pval_threshold: float = 0.05,
) -> tuple[list, dict]:
    """Engle-Granger cointegration screen over SSD-pre-ranked candidate pairs.

    Testing all N(N-1)/2 pairs with EG is prohibitively slow for large
    universes, so the standard two-stage approach is used: pre-rank by SSD
    (distance method), then run the EG test only on the top candidates.

    For each candidate pair the two-step Engle-Granger procedure is applied
    to LOG prices over the formation window:

        Stage 1:  ln(P_A) = α + β·ln(P_B) + ε   (OLS)
        Stage 2:  ADF test on the residual ε̂   (statsmodels coint)

    Pairs with p-value < pval_threshold qualify; the n pairs with the
    smallest p-values are returned.

    Parameters
    ----------
    prices_window  : RAW price DataFrame for the formation window.
    candidates     : list of (ticker1, ticker2, ssd) from select_top_pairs.
    n              : number of pairs to return (default 20, as in GGR Top-20).
    pval_threshold : EG p-value cutoff (default 0.05).

    Returns
    -------
    selected  : list of (ticker1, ticker2, pvalue), best p-values first.
    eg_params : dict (t1, t2) → {beta, alpha, sigma_eg, pvalue}; sigma_eg is
                the formation-period std of the EG residual — the locked σ
                used by CointStrategy.
    """
```

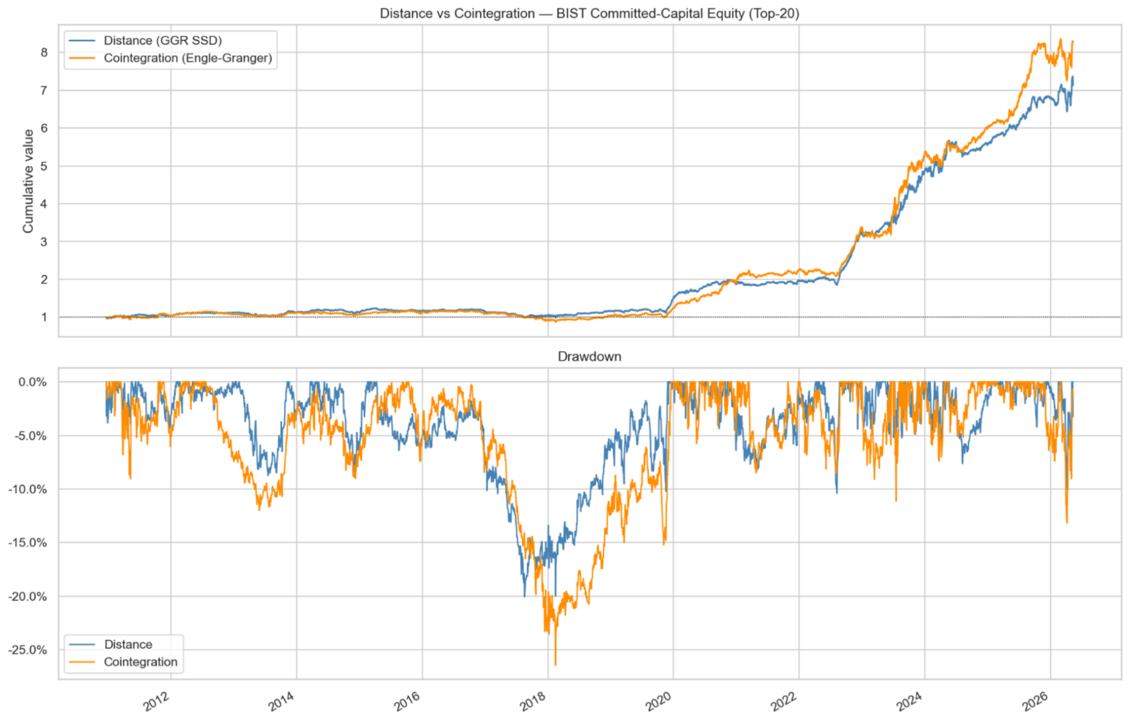
Instead of raw price distance, we fit a linear regression between the two stock prices $Pa = \boldsymbol{\alpha} + \boldsymbol{\beta P}B + \boldsymbol{\epsilon}$

- We calculate the rolling Z-score of the regression residuals (error term).

Signals:

- Open position when the residual Z-score crosses +2 or -2 (the spread is mispriced)
- Close position when it converges back to 0 (equilibrium).

Distance vs Cointegration



Both methods are tested on BIST Top-20 pairs over 2011–2026. Cointegration slightly outperforms in cumulative return but at the cost of higher drawdown risk.

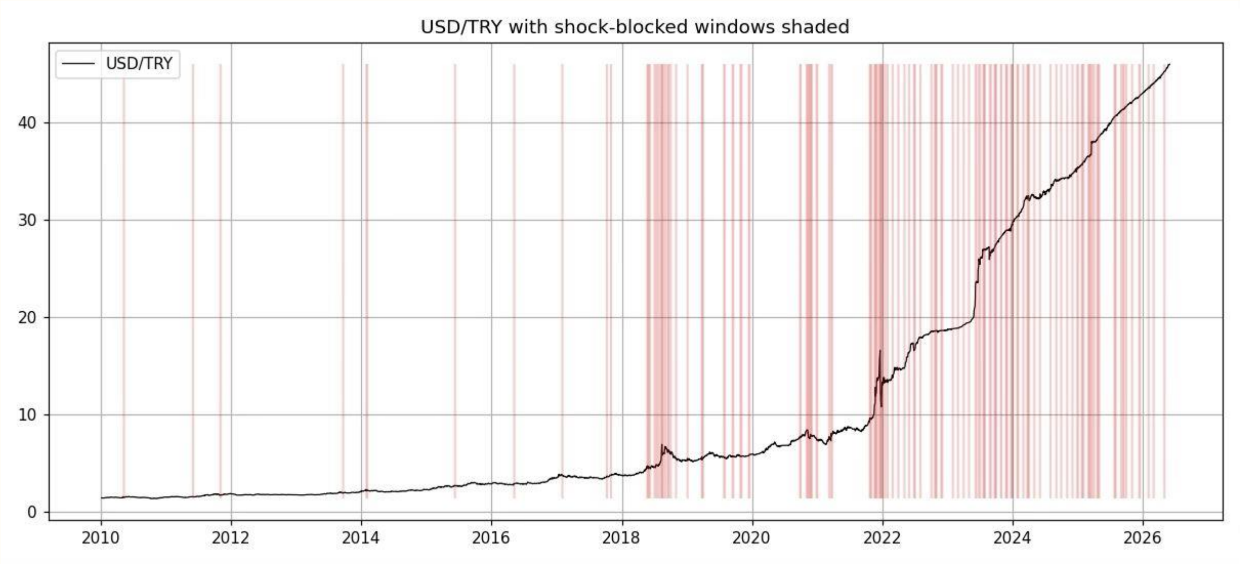
- Both methods track each other closely over 2011–2026, with returns accelerating sharply post-2020.
- Cointegration suffers a deeper drawdown (~—22%) during the 2018 lira crisis vs. Distance (~—16%).
- Cointegration delivers higher cumulative returns; Distance offers a more stable equity curve.

Performance Metrics Comparison: Top-20 Pairs (2011-2026)

Performance Metric	Distance Approach (GGR SSD)	Cointegration Approach (EG)
Net P/L (Committed Capital)	613%	**727%**
Annualized Return	13.1%	**14.1%**
Annualized Volatility	**10.3%**	11.4%
Sharpe Ratio	**1.27**	1.24
Maximum Drawdown	**~-16.0%**	~-22.0%

Risk & Robustness: The Distance method uses a fixed historical threshold $\boldsymbol{\sigma}$, keeping volatility low and the equity curve smooth (better Sharpe ratio). In contrast, Cointegration relies on a moving regression coefficient $\boldsymbol{\beta}$ that breaks down during sudden market shocks (like in 2018), leading to deeper drawdowns.

Improvement Idea 1: TCMB-Policy Shock Filter

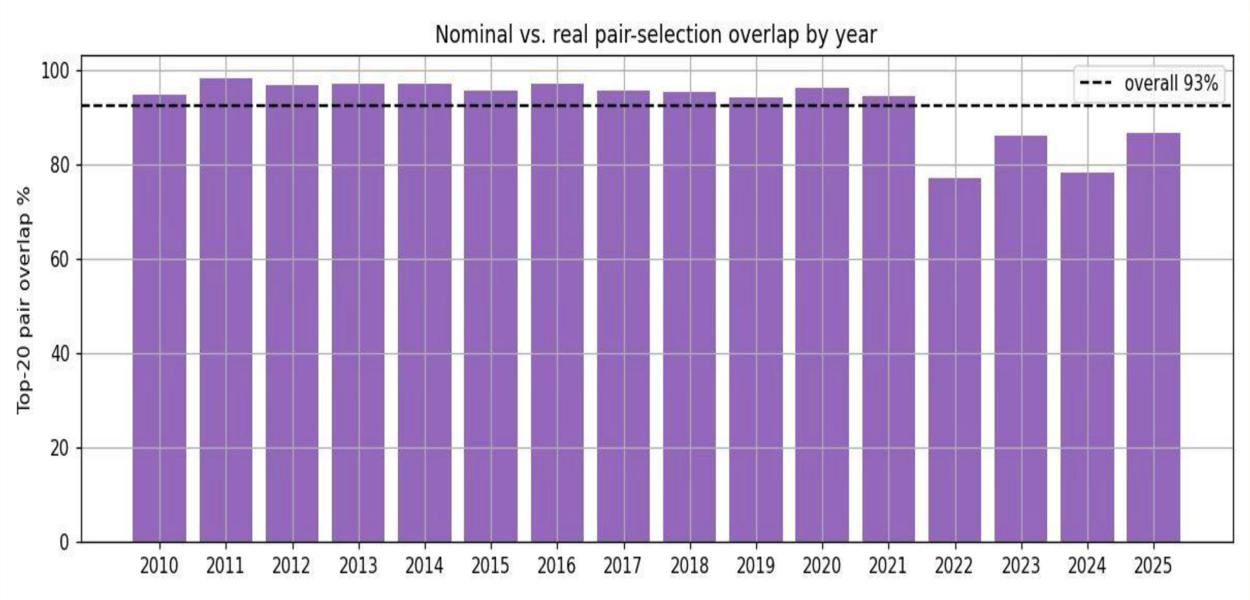


Concept

- Block NEW entries during TCMB rate surprises, >3% USD/TRY days, or high CPI prints. Open positions run to normal exit.

- Real data: 30 surprise rate decisions, USD/TRY, TÜİK CPI. 11.5% of days blocked.
- Result: Sharpe −0.39 → −0.35, Max DD 69% → 66%, −213 trades.

Idea 2–Real (CPI-Adjusted) Pair Formation

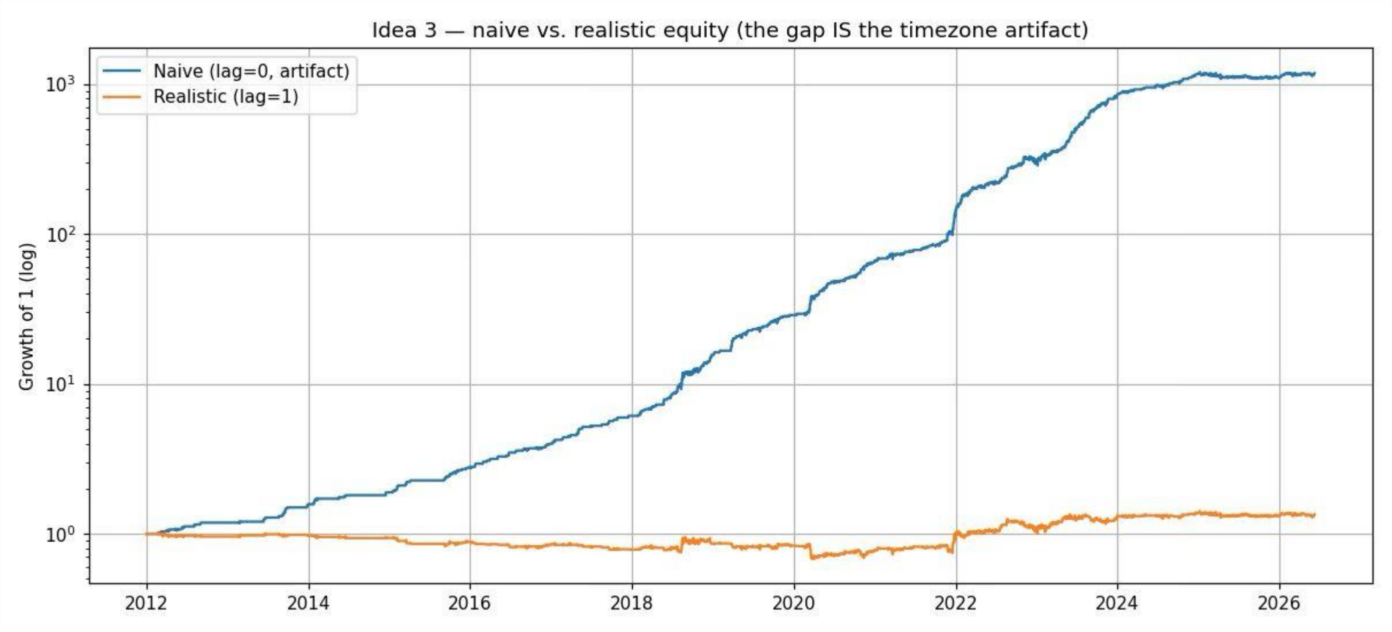


Hypothesis

- Idea: inflation creates ‘phantom’ pairs; deflate prices by CPI to keep only real co-movement.
- Math finding: CPI is common to all stocks, so it factors out of the spread — deflation only TIME-WEIGHTS the SSD (down-weights recent days), it does NOT remove an inflation trend.
- Result: collapses on full set (Sharpe —0.39 → —0.43).

Negative Results

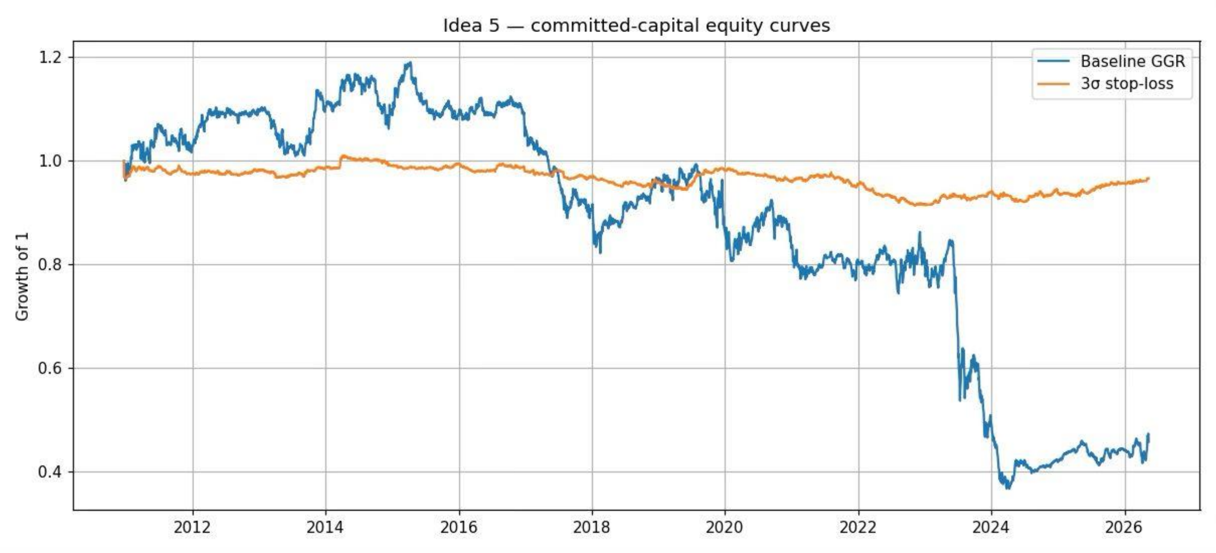
Idea 3–Cross-Listing Arb (TCELL.IS vs TKC)



Concept

- Same firm, two markets: BIST TCELL.IS vs NYSE ADR TKC. Empirical ratio 2.445 ≈ documented 2.5.
- Trap: naive backtest looks amazing: Sharpe 3.7, 99% win. but it is a TIMEZONE artifact (BIST closes 5h before NYSE → stale-price fake reversion).
- Realistic (next-bar execution): edge collapses to Sharpe ~0.3 with high FX risk.

Idea 4 – 3σ Stop-Loss



Concept

- GGR has no stop. Do & Faff (2010) suggests that a pair that reaches 3σ is structurally broken.
- One rule added: close at 3σ for a loss, and stop re-trading that pair.

Stop threshold sensitivity

2.5σ Sharpe —0.07 | 3.0σ —0.11 | **3.5σ +0.15**

Max DD 69% → 10%

3.5σ turns Sharpe POSITIVE (+0.15)

Data Limitations

- **Survivorship bias:** Yahoo Finance only serves surviving tickers. We manually added delisted names (ROYAL, YGYO, ...) from EIKONS spreadsheets. (Thanks to our TA!)
- We cannot measure the real spread, so we approximate trading cost two ways — a one-day execution delay (paper's Panel B) and an explicit per-trade commission (0/5/10/20 bps).
- Lack of exhaustive list of short-selling restrictions.
- **Implication:** Results are directionally trustworthy but not execution-ready; a real deployment needs clean, point-in-time, intraday data.